Learned Warm Starts and 8 bit Inference for THz MIMO-SAR Inverse Simulation: A Synthetic Study

Md Forhadul Islam
London, United Kingdom
forhad.sde@gmail.com
Short independent study based on the public InverseThzSim code; not submitted for publication

***Abstract*: Saleh et al. showed that object parameters can be estimated directly from raw MIMO-SAR data at 235 to 270 GHz by inverse simulation, without forming an image. Each estimate still needs many forward simulations, and the optimizer starts from a fixed initial value. In this short study I trained a small convolutional network on 5100 measurements simulated with their public code and used its output as the starting point of their optimization. On held out simulated data the network estimates the lateral position of a sphere with a mean absolute error of 2.8 mm and the distance of its front surface with 1.2 mm, in 0.36 ms. Depth and radius on their own are hard to separate. After INT8 quantization the errors change by less than 0.1 mm while the weights become four times smaller. With the original optimizer settings, the first steps move the estimate away from the good starting point again. When I reduced the step size to match the error of the network, the median lateral error after the last of 30 iterations was 0.38 mm, compared with 2.80 mm for the original cold start. All results are on simulated data only.**

***Index Terms*: THz imaging, MIMO-SAR, inverse simulation, differentiable rendering, warm start, quantization**

I. INTRODUCTION

Millimeter wave and THz MIMO-SAR systems are attractive for nondestructive testing and security screening. Saleh *et al.* [1] recently proposed to estimate object parameters directly from the raw measurement instead of first reconstructing an image, for example by back projection [2], and then searching that image. A differentiable ray tracer predicts the measurement of a parametrized scene, and the scene parameters are optimized until the simulated and the measured data agree. This removes the reconstruction step and gives an estimate with a clear physical meaning [1].

Two things caught my attention when I ran their code. Every optimization step needs a few full simulations, about 4 s each on my GPU, and the search for the lateral position always starts at x = 0. The loss of radar data oscillates with the position, and [1] handles this by smoothing the loss over the parameter space. I wanted to know whether a good starting point from a learned estimator helps this optimization, and whether such an estimator still works at 8 bit precision, which matters for running it next to the sensor.

The idea of using a network output to initialize a physics based fit is not new. Wong *et al.* [3] trained an encoder to predict THz model parameters and used its predictions to start a local optimizer, which converged about twice as fast. Here I apply the same idea to raw data inverse simulation as in [1]. My contributions are small and specific:

(i) an estimator trained only on data from the public simulator of [1], mapping raw MIMO-SAR data to sphere position and radius;
(ii) a comparison of cold and warm started position search, first with the settings of [1] unchanged and then with a step size matched to the estimator error;
(iii) an INT8 post training quantization test of the estimator; and
(iv) Jupyter notebooks with all outputs, available at github.com/forhadsde/THz_MIMO_SAR.

II. RELATED WORK

Inverse simulation for radar. Inverse rendering recovers scene parameters by matching a differentiable simulation to measurements. Hofmann *et al.* [4] used it for near field mmWave MIMO radar at 72 to 82 GHz to reconstruct material properties, with the scene geometry taken from additional optical sensors. Saleh *et al.* [1] estimate geometry from raw MIMO-SAR data alone and, on measured data, report mean errors of 0.11 mm for the lateral position of a 40 mm sphere, 0.3 mm for its height and 0.35 mm for sphere radii, at least an order of magnitude better than back projection. Their optimizer smooths the loss with Gaussian perturbations of the parameters, following Fischer and Ritschel [5].

Learning for THz parameter estimation. Besides [3], Wong *et al.* used deep networks as optimization priors for THz model parameter estimation [6]. Simulation based inference in general trains networks on simulated data to amortize an inverse problem [7].

Efficient inference. Integer quantization with per channel weight scales and calibrated activation ranges is a common way to run networks on low power hardware [8], [9]. Quantization aware training with a straight through estimator [10], [11] and knowledge distillation [12] can recover accuracy when post training quantization is not enough.

III. METHOD

A. Measurement Model and Inverse Simulation of [1]

I use the system model and simulator of [1] without changes. The hardware has a linear array of 12 transmitters and 32 receivers; as in [1], 10 transmitters and 31 receivers enter the loss (in the released code one transmitter and one receiver are switched off and the loss uses transmitter rows 1 to 10). Each pair is measured at 64 frequencies between 235 and 270 GHz. For scene parameters β the simulator traces 10^6 paths per

transmitter towards the object with Mitsuba 3 [13], [14], using single bounce Phong reflectance and Gaussian antenna patterns, and sums the complex contribution of every path. A random starting phase per transmitter and frequency is applied and removed again with a simulated calibration plane. The amplitudes of each transmitter receiver pair are normalized along frequency, and the loss of [1] compares Hamming windowed range profiles,

$$L(\beta) = \Sigma_{t,r} \| |\mathscr{F}(w \odot \hat{S}_{t,r}(\beta))| - |\mathscr{F}(w \odot \hat{M}_{t,r})| \|_2^2 \qquad (1)$$

where $\hat{S}$ and $\hat{M}$ are the normalized simulated and reference signals, w is the Hamming window and $\mathscr{F}$ the discrete Fourier transform. The gradient of the smoothed loss is estimated with importance sampled, antithetic perturbations [5], and the parameters are updated with Adam [15]. The released notebook for position search uses a kernel standard deviation of 12.5 mm, a learning rate of 20 mm and one antithetic pair per step; I use these values unless stated otherwise. In the scene convention of the released code a sphere set to position z with radius r has its center at z − r, so the surface facing the array is at z − 2r. I report this front surface distance separately below.

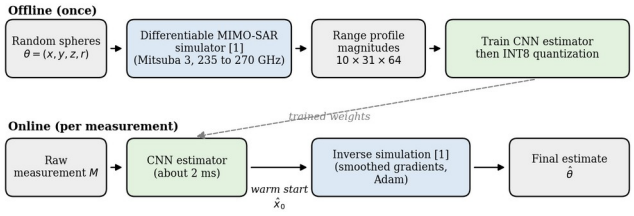


Fig. 1. Overview. Offline, a CNN is trained on simulated data from [1] and quantized. Online, its estimate is the starting point of the inverse simulation of [1], which gives the final estimate.

B. Estimator

The estimator sees the magnitude of the windowed range profiles of transmitters 1 to 10 and all 31 receivers, a 10 × 31 × 64 input. Unlike the loss, I normalize the whole measurement once by its maximum instead of each pair separately. The network has three blocks of convolution, batch normalization [16] and max pooling, a 256 unit hidden layer and a sigmoid output for (x, y, z, r) scaled to the training ranges, with 883428 parameters. I trained it for 60 epochs with a smooth L1 loss, AdamW and a one cycle schedule, with light noise added to the inputs, and kept the checkpoint with the lowest validation error. Because the noise changes the input statistics, I recomputed the batch normalization statistics on clean training inputs at the end. In a first training run without this step I saw a constant offset of about 3 mm in z and r; in the final run the remaining offset is 1.1 mm in x, −1.8 mm in z and −0.4 mm in r.

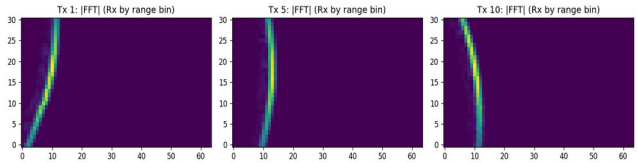


Fig. 2. Estimator input for one simulated sphere: normalized range profile magnitudes of three transmitters (receivers against range bins).

C. Warm Start

I follow the one dimensional position search of [1]: only x is optimized, while y, z and r are set to their true values. The cold start begins at $x_0 = 0$ as in the released notebook, the warm start begins at the estimator output $\hat{x}$. In the first warm start variant nothing else changes. Adam's first steps are about one learning rate long, here 20 mm, so a start that is already within a few millimeters can be lost again. In the second variant I therefore set both the kernel standard deviation and the learning rate to 3 mm, about the lateral error of the estimator. I did not tune the cold start in the same way, so this variant shows what a good starting point allows, not a tuned comparison.

D. 8 bit Quantization

I folded batch normalization into the convolutions, quantized the weights per output channel to symmetric INT8 and the activations per tensor to INT8, with ranges taken from 512 training samples. This is simulated quantization in PyTorch. Quantization aware training was planned as a fallback but was not needed for INT8. As a stress test I also quantized the weights to INT4 without retraining.

IV. EXPERIMENTAL SETUP

All runs used the public code of [1] on one NVIDIA RTX 3090. The authors built Mitsuba 3.3.0 from source with monochromatic variants. The binary release of the same version only contains RGB variants, so I used *cuda_ad_rgb*. One simulation with calibration took 4.0 s.

Data. I sampled spheres uniformly with x in [−120, 120] mm, y in [20, 60] mm, z in [310, 350] mm and r in [8, 22] mm, which covers the measured positions and radii in [1]. I simulated 5100 measurements and split them 90/5/5 with a fixed seed into 4590 training, validation and test samples. The test set was only used for the final numbers. The measured data of [1] is published on IEEE DataPort [17], but I did not have access to it for this study.

Reproduction check. Before any learning I ran the cold started search of [1] on a simulated sphere at the first measured position (x = −107.75 mm, r = 20 mm). After 50 iterations the best estimate was 0.22 mm from the true value, and the error first fell below 1 mm after 20 iterations (Fig. 3).

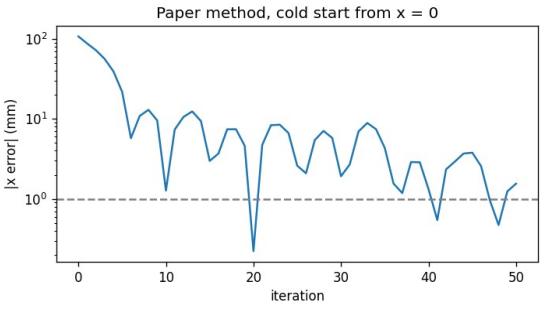


Fig. 3. Reproduction of the cold started position search of [1] on a simulated sphere.

V. RESULTS

A. Estimator

Table I lists the test errors. The lateral error drops from 58.4 mm for a predictor that always outputs the training mean to 2.8

mm. The z and r errors are larger on their own, but they are almost perfectly correlated (correlation 0.99). Their combination z − 2r, the front surface distance, is estimated with 1.2 mm, compared with 12.1 mm for the mean predictor and a nominal range resolution of c/(2B) ≈ 4.3 mm for the 35 GHz bandwidth. So the network finds where the reflecting surface is, but separating size from distance with magnitude features alone is ill conditioned for these small spheres. The vertical coordinate y is not better than the mean, which I expect for magnitude range profiles from two parallel linear arrays. One prediction takes 0.36 ms on the GPU.

TABLE I
TEST MEAN ABSOLUTE ERROR (MM) ON HELD OUT SIMULATED DATA

Model	x	y	z	r	z − 2r
Mean predictor	58.4	10.2	10.2	3.7	12.1
CNN, FP32	2.8	10.2	5.1	2.6	1.2
CNN, INT8	2.9	10.2	5.1	2.6	1.2
CNN, INT4 weights	3.4	10.2	5.1	2.6	n/a

255 test samples. Weight memory of convolution and linear layers: 3447 kB (FP32), 862 kB (INT8), 431 kB (INT4).

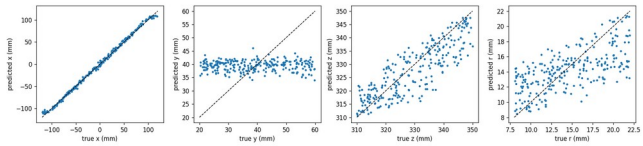


Fig. 4. Predicted against true parameters on the test set (x, y, z, r from left to right).

B. Warm Start

Table II and Fig. 5 compare the three starts on the same 6 test spheres with 30 iterations each. On these spheres the estimator alone is between 0.3 and 2.0 mm from the true position. With the settings of [1] this advantage is mostly lost within a few iterations: after 5 iterations the median error is 9.2 mm for the warm start and 11.9 mm for the cold start, and at the last iteration 3.01 mm and 2.80 mm. With the 3 mm step size the warm start stays close to the solution, with a median error of 1.17 mm after 5 iterations and 0.38 mm at the last iteration, lower than the cold start in 6 of 6 cases. One iteration took about 11 s, so the useful saving is in iterations needed to stay near the solution, not in the cost of each iteration.

Like [1], I also report the iterate with the lowest loss. Because my reference measurement comes from the same noise free simulator, this iterate always coincided with the lowest true error, and its median error (0.75, 0.24 and 0.016 mm) is much smaller than at the last iteration. I do not see these sub millimeter values as a realistic precision; with measured data they would be limited by noise and model errors.

TABLE II
LATERAL POSITION SEARCH WITH THREE STARTS

Start	Median after 0 / 5 / 20 it.	Last iteration, median (IQR)	Lowest loss iterate, median
Cold, settings of [1]	36.9 / 11.9 / 4.5	2.80 (1.4 to 4.9)	0.75
Warm, settings of [1]	1.0 / 9.2 / 4.2	3.01 (1.0 to 4.8)	0.24
Warm, σ = lr = 3 mm	1.0 / 1.2 / 0.5	0.38 (0.3 to 0.5)	0.016

Errors in mm, 6 test spheres, 30 iterations. IQR: interquartile range.

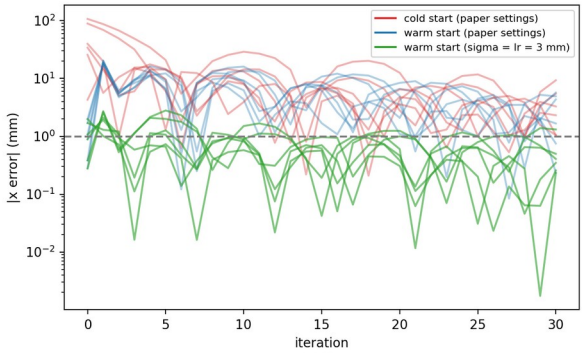


Fig. 5. Lateral position error per iteration for the cold start of [1] (red), the warm start with the settings of [1] (blue) and the warm start with a 3 mm step size (green).

C. 8 bit Inference

With INT8 weights and activations the test errors change by at most 0.1 mm, and the front surface error stays at 1.2 mm (Table I), while the weights take a quarter of the memory. With INT4 weights and no retraining the lateral error rises from 2.80 to 3.35 mm, so INT8 is enough here and INT4 would need quantization aware training [11].

VI. LIMITATIONS

This study has clear limits. All data comes from the simulator of [1], so the estimator has never seen real noise or calibration errors, and the reference and the estimate share the same simulator; the measured data [17] is the honest next test. Only spheres were used. The warm start test covers 6 spheres, one training seed and a one dimensional search with y, z and r known, and the cold start was not tuned. The checkpoint was selected on validation error from a single run, and the validation error varied a lot between epochs. The quantization results describe accuracy only, not speed or power on hardware. Finally, I used the RGB instead of the monochromatic Mitsuba variant and ran 30 instead of 50 iterations, so my cold start numbers are not directly comparable with the measured results of [1].

VII. CONCLUSION

A small network trained only on simulated data can give a useful starting point for the inverse simulation of [1] in well under a millisecond, and it keeps its accuracy at INT8. What surprised me most is that a good starting point helps only when the optimizer is allowed to take steps of a matching size; with the original step size it is soon lost. I would like to understand how this behaves on the measured data of [1] and in a search over all four parameters, and I would be grateful for any thoughts on this direction.

Acknowledgment. I thank M. Saleh, M. Kahl, P. Haring Bolívar and A. Kolb for making their InverseThzSim code publicly available.

REFERENCES

[1] M. Saleh, M. Kahl, P. Haring Bolívar, and A. Kolb, "Direct detection of object parameters from raw data in MIMO-SAR imaging at 235–270

GHz via inverse simulation," *IEEE J. Microw.*, vol. 6, no. 1, pp. 126–136, Jan. 2026, doi: 10.1109/JMW.2025.3627070.

[2] D. M. Sheen, D. L. McMakin, and T. E. Hall, "Three-dimensional millimeter-wave imaging for concealed weapon detection," *IEEE Trans. Microw. Theory Techn.*, vol. 49, no. 9, pp. 1581–1592, Sep. 2001.

[3] T. M. Wong, M. Kahl, P. Haring Bolívar, A. Kolb, and M. Möller, "Training auto-encoder-based optimizers for terahertz image reconstruction," in *Proc. German Conf. Pattern Recognit. (GCPR)*, LNCS vol. 11824, 2019, pp. 93–106.

[4] N. Hofmann, V. Wirth, J. Bräunig, I. Ullmann, M. Vossiek, T. Weyrich, and M. Stamminger, "Inverse rendering of near-field mmWave MIMO radar for material reconstruction," *IEEE J. Microw.*, vol. 5, no. 2, pp. 356–372, 2025, doi: 10.1109/JMW.2025.3535077.

[5] M. Fischer and T. Ritschel, "Plateau-reduced differentiable path tracing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 4285–4294.

[6] T. M. Wong, H. Bauermeister, M. Kahl, P. Haring Bolívar, M. Möller, and A. Kolb, "Deep optimization prior for THz model parameter estimation," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2022, pp. 3811–3820.

[7] K. Cranmer, J. Brehmer, and G. Louppe, "The frontier of simulation-based inference," *Proc. Natl. Acad. Sci.*, vol. 117, no. 48, pp. 30055–30062, 2020.

[8] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.

[9] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv:2106.08295, 2021.

[10] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

[11] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[12] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[13] W. Jakob *et al.*, "Mitsuba 3 renderer," version 3.3.0, 2022. [Online]. Available: https://mitsuba-renderer.org

[14] W. Jakob, S. Speierer, N. Roussel, and D. Vicini, "Dr.Jit: A just-in-time compiler for differentiable rendering," *ACM Trans. Graph.*, vol. 41, no. 4, 2022, doi: 10.1145/3528223.3530099.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[16] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

[17] M. Saleh, "Direct detection of objects and parameters from raw data in MIMOSAR imaging dataset," IEEE DataPort, doi: 10.21227/8jb1-x962.